

Hints and Solutions Sheet Module 3

Section A

Problem 1

```
import numpy as np
import matplotlib.pyplot as plt

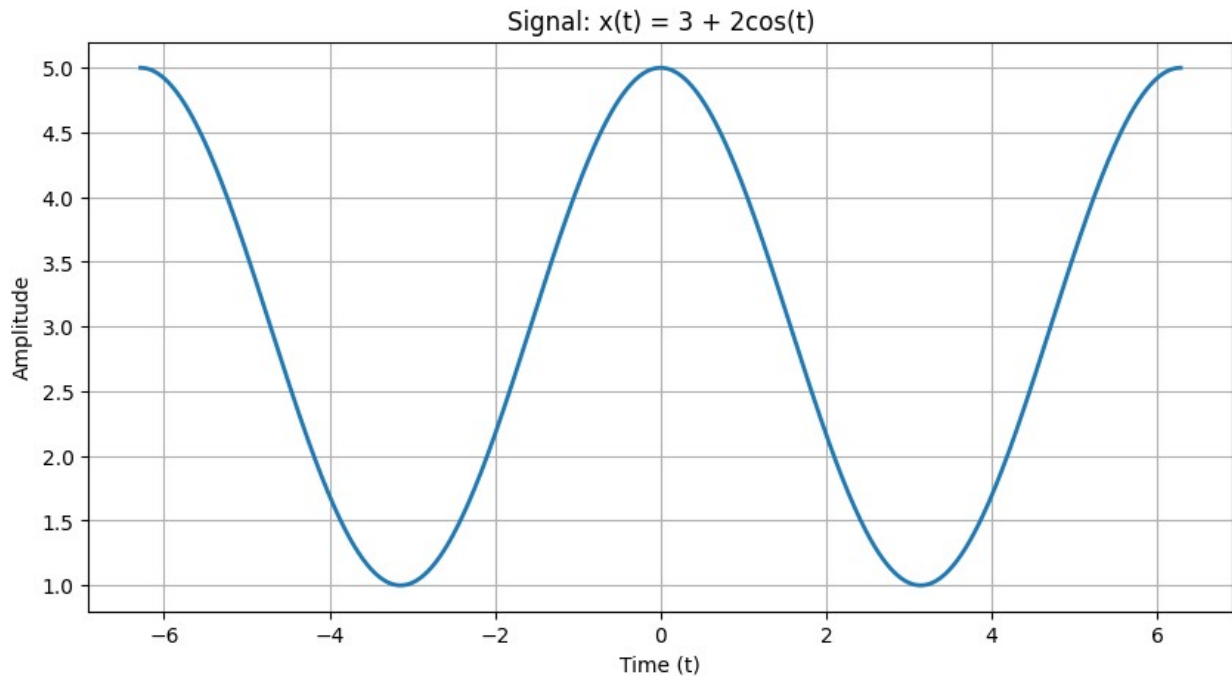
# Time vector
t = np.linspace(-2*np.pi, 2*np.pi, 1000)

# Signal
x = 3 + 2*np.cos(t)

# Plot
plt.figure(figsize=(10,5))
plt.plot(t, x, linewidth=2)
plt.title("Signal:  $x(t) = 3 + 2\cos(t)$ ")
plt.xlabel("Time (t)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

# Calculations
DC = 3
Amplitude = 2
Period = 2*np.pi

print("DC Component =", DC)
print("Amplitude =", Amplitude)
print("Fundamental Period =", Period)
```



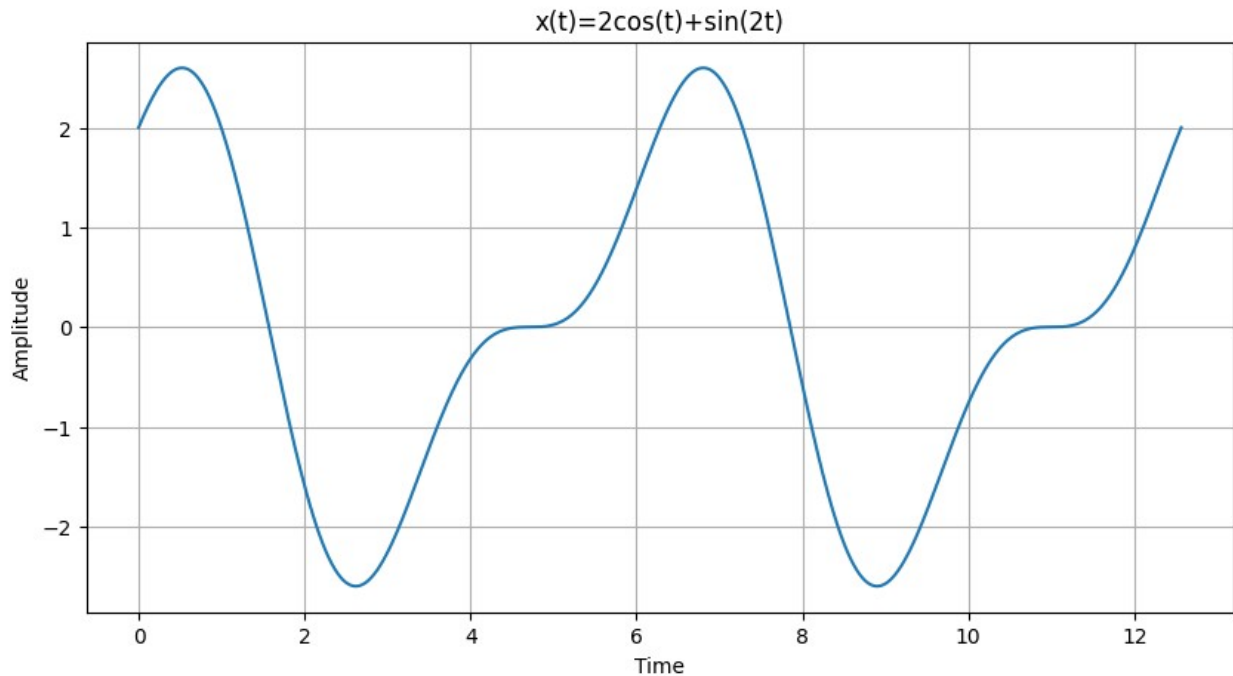
DC Component = 3
Amplitude = 2
Fundamental Period = 6.283185307179586

Problem 2

```
t = np.linspace(0,4*np.pi,1000)
x = 2*np.cos(t) + np.sin(2*t)

plt.figure(figsize=(10,5))
plt.plot(t,x)
plt.title("x(t)=2cos(t)+sin(2t)")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

print("Harmonics Present:")
print("First Harmonic : cos(t)")
print("Second Harmonic : sin(2t)")
print("Fundamental Period = 2π")
```



Harmonics Present:
 First Harmonic : $\cos(t)$
 Second Harmonic : $\sin(2t)$
 Fundamental Period = 2π

Problem 3

Using Euler's Identity:

$$\cos(t) = \frac{e^{jt} + e^{-jt}}{2}$$

Fourier Series Coefficients $C_1 = 0.5$

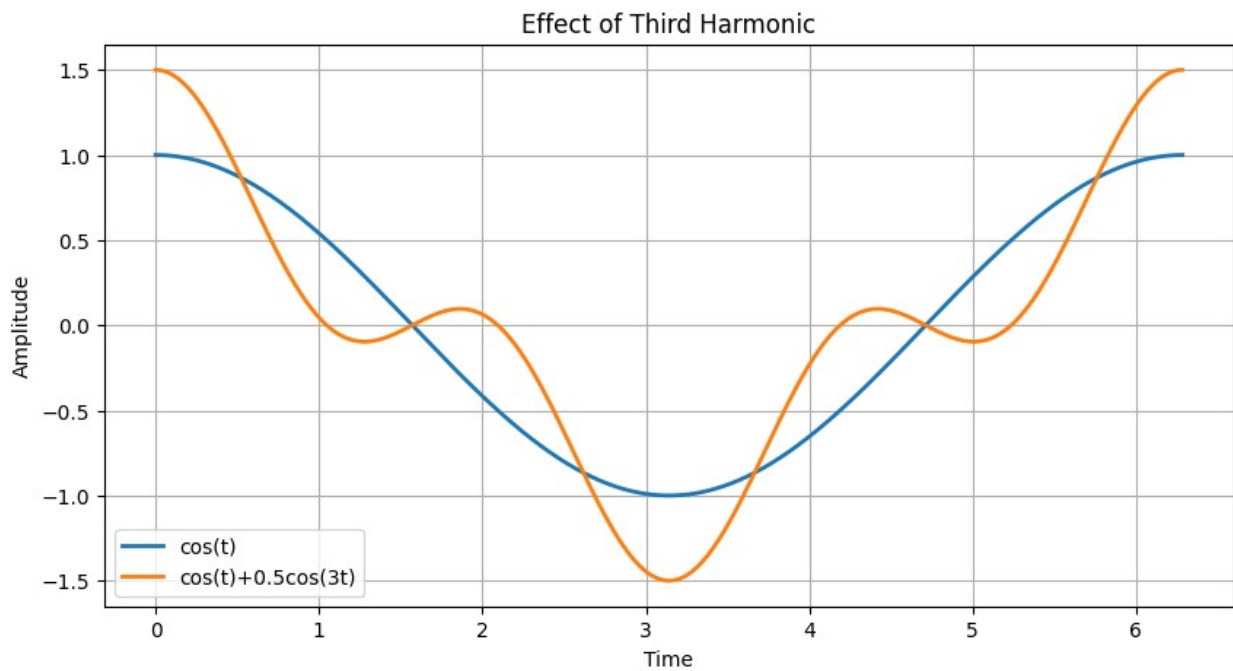
$$C_{-1} = 0.5$$

All other coefficients = 0

Problem 4

```
t = np.linspace(0, 2*np.pi, 1000)
x1 = np.cos(t)
x2 = np.cos(t) + 0.5*np.cos(3*t)
plt.figure(figsize=(10, 5))
plt.plot(t, x1, label="cos(t)", linewidth=2)
plt.plot(t, x2, label="cos(t) + 0.5cos(3t)", linewidth=2)
```

```
plt.legend()
plt.grid(True)
plt.title("Effect of Third Harmonic")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.show()
```



Section B

Problem 5

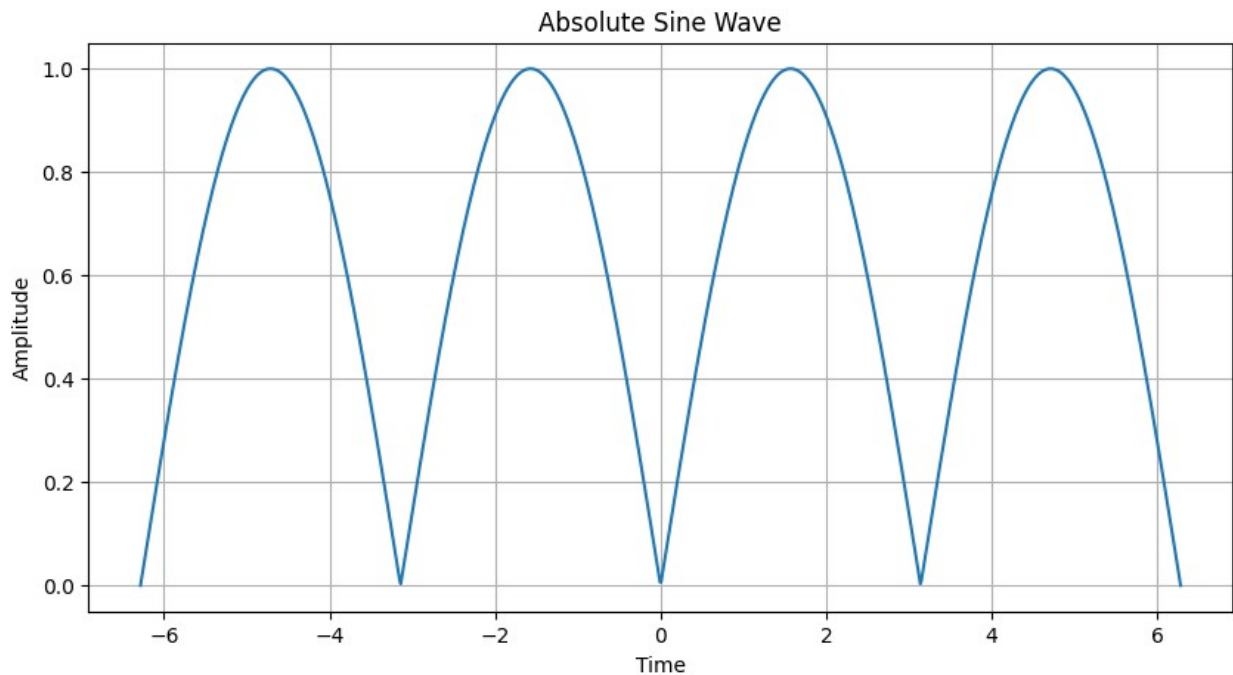
```
t = np.linspace(-2*np.pi, 2*np.pi, 1000)
x = np.abs(np.sin(t))
plt.figure(figsize=(10, 5))
plt.plot(t, x)
plt.title("Absolute Sine Wave")
plt.xlabel("Time")
```

```

plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

print("Verification of Dirichlet Conditions")
print("1. Periodic : Yes")
print("2. Finite Number of Maxima/Minima : Yes")
print("3. Finite Number of Discontinuities : None")
print("4. Absolutely Integrable over one period : Yes")

```



```

Verification of Dirichlet Conditions
1. Periodic : Yes
2. Finite Number of Maxima/Minima : Yes
3. Finite Number of Discontinuities : None
4. Absolutely Integrable over one period : Yes

```

Problem 6

```

import numpy as np
import matplotlib.pyplot as plt

# Time axis

```

```

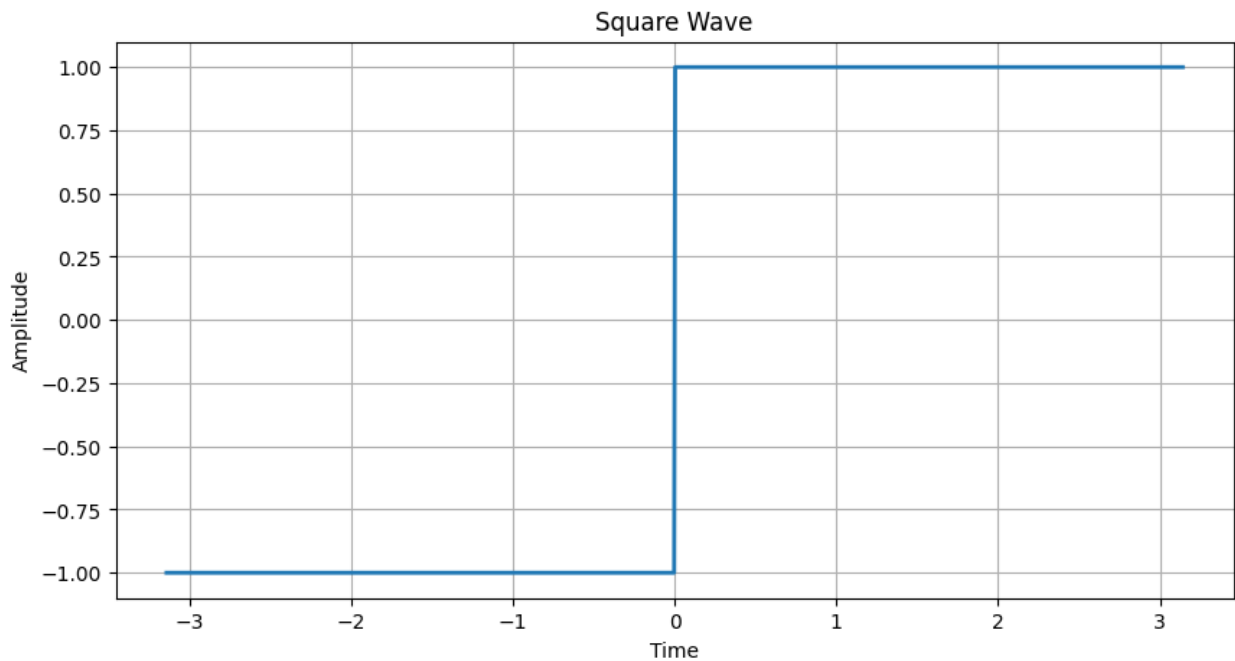
t = np.linspace(-np.pi, np.pi, 1000)

# Square wave
x = np.where(np.sin(t) >= 0, 1, -1)

# Plot
plt.figure(figsize=(10,5))
plt.plot(t, x, linewidth=2)
plt.title("Square Wave")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

print("Dirichlet Conditions Verification")
print("Periodic           : Yes")
print("Finite discontinuities : Yes")
print("Finite maxima/minima   : Yes")
print("Absolutely integrable  : Yes")

```



```

Dirichlet Conditions Verification
Periodic           : Yes
Finite discontinuities : Yes
Finite maxima/minima   : Yes
Absolutely integrable  : Yes

```

Problem 7

```
t = np.linspace(-np.pi, np.pi, 1000)

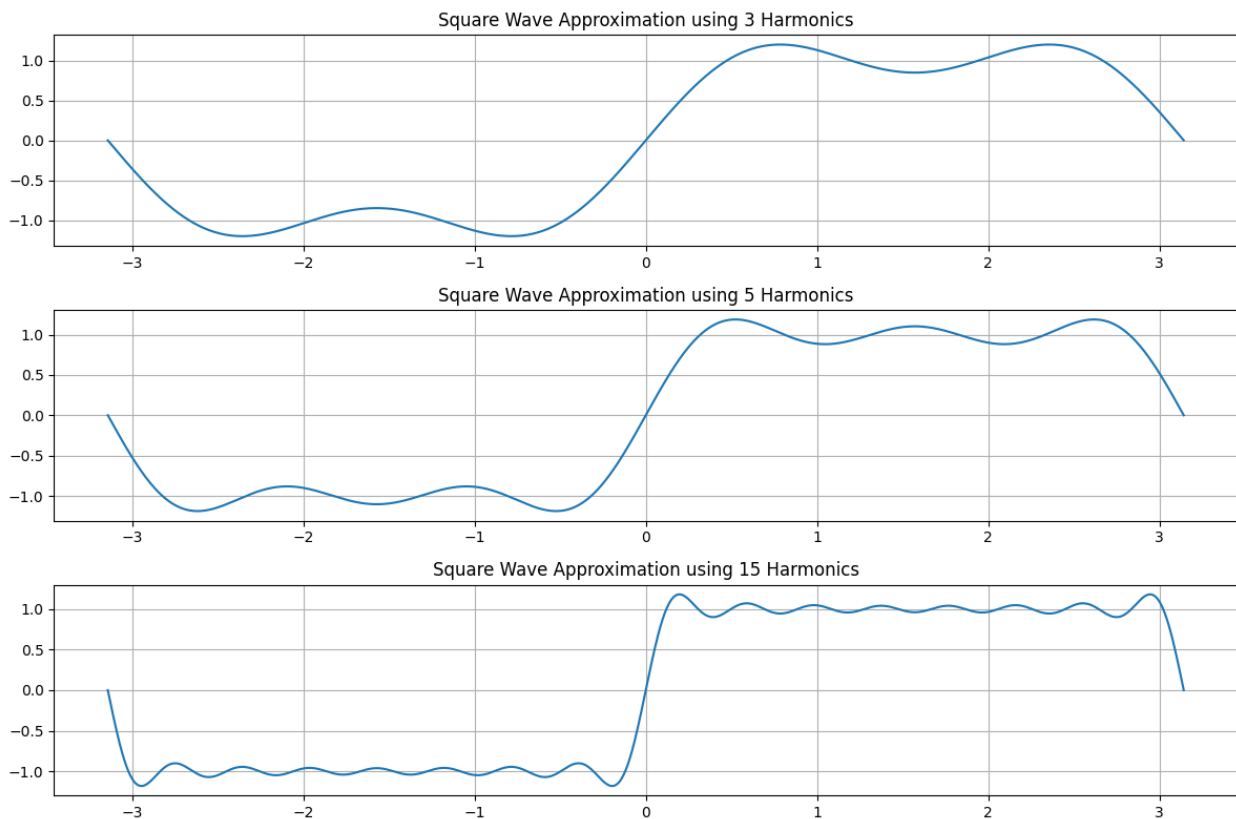
plt.figure(figsize=(12,8))

harmonics = [3,5,15]

for i,N in enumerate(harmonics):
    x = np.zeros_like(t)
    for n in range(1,N+1,2):
        x += (4/(np.pi*n))*np.sin(n*t)

    plt.subplot(3,1,i+1)
    plt.plot(t,x)
    plt.title(f"Square Wave Approximation using {N} Harmonics")
    plt.grid(True)

plt.tight_layout()
plt.show()
```

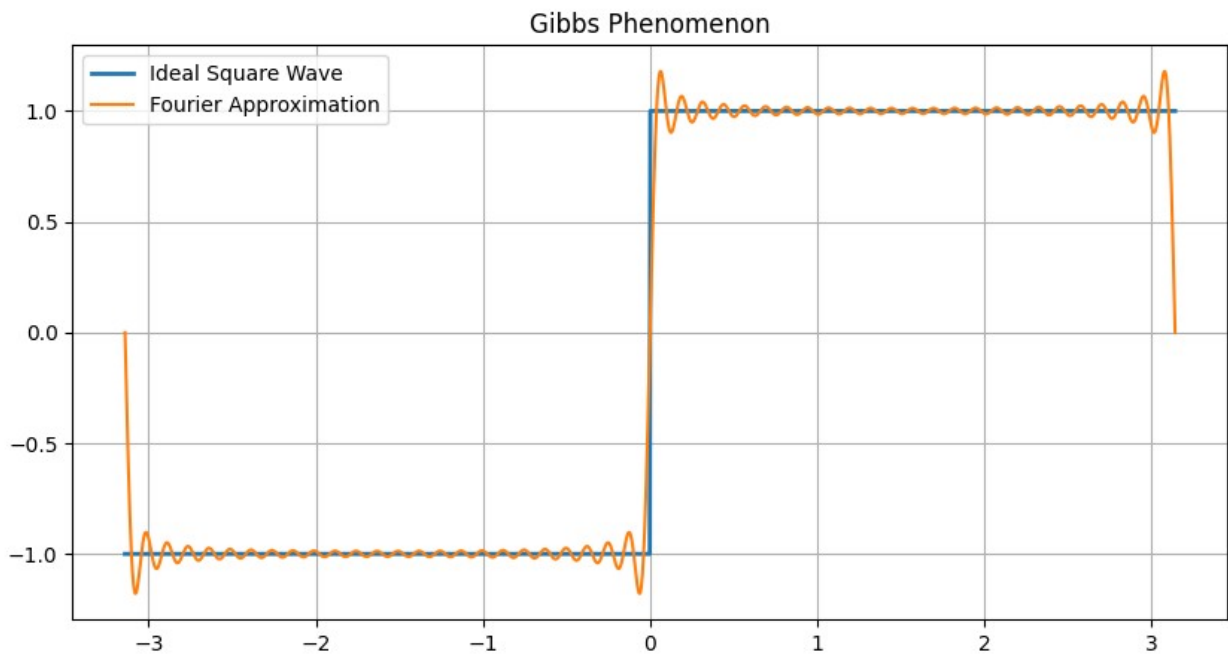


Problem 8

```
t = np.linspace(-np.pi,np.pi,2000)
x = np.zeros_like(t)

for n in range(1,51,2):
    x += (4/(np.pi*n))*np.sin(n*t)

ideal = np.where(np.sin(t)>=0,1,-1)
plt.figure(figsize=(10,5))
plt.plot(t,ideal,label="Ideal Square Wave",linewidth=2)
plt.plot(t,x,label="Fourier Approximation")
plt.legend()
plt.grid(True)
plt.title("Gibbs Phenomenon")
plt.show()
```

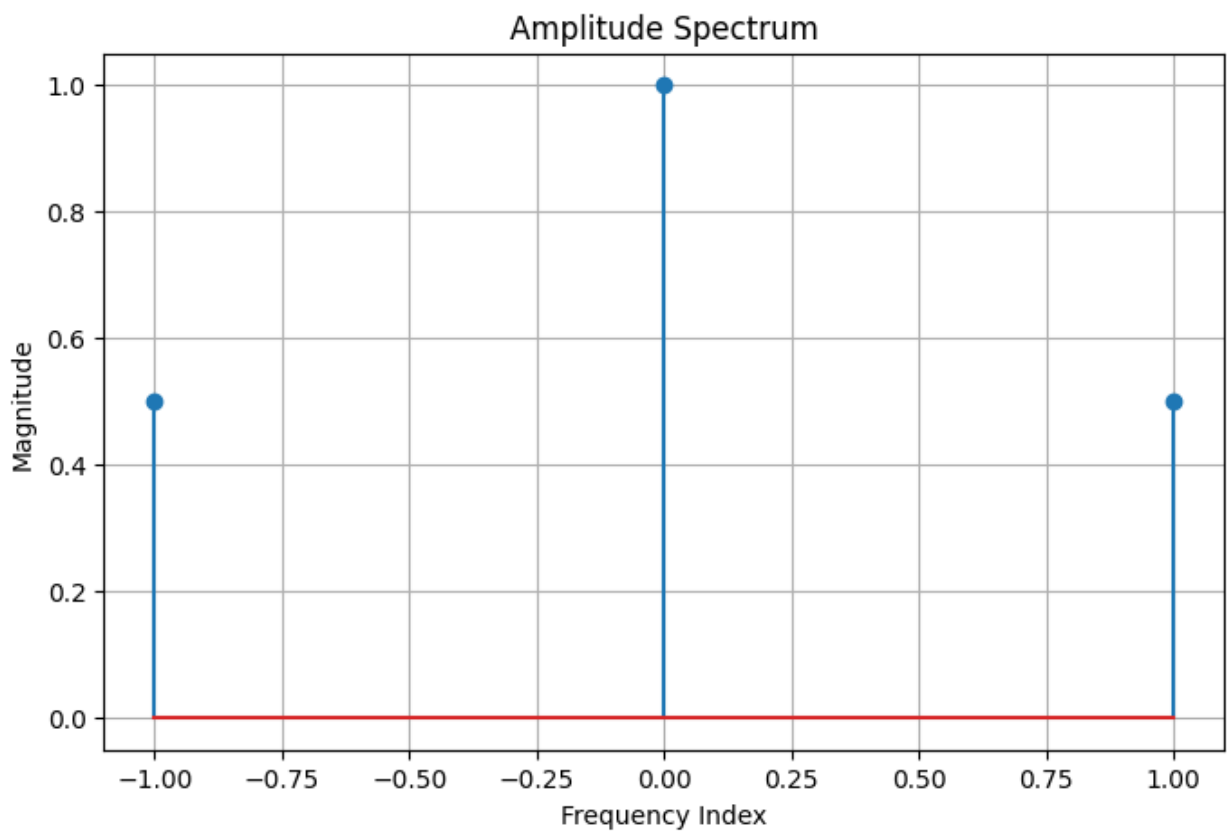


Problem 9

```
frequency = np.array([-1,0,1])
amplitude = np.array([0.5,1,0.5])
```



```
plt.figure(figsize=(8,5))
plt.stem(frequency,amplitude)
plt.title("Amplitude Spectrum")
plt.xlabel("Frequency Index")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```



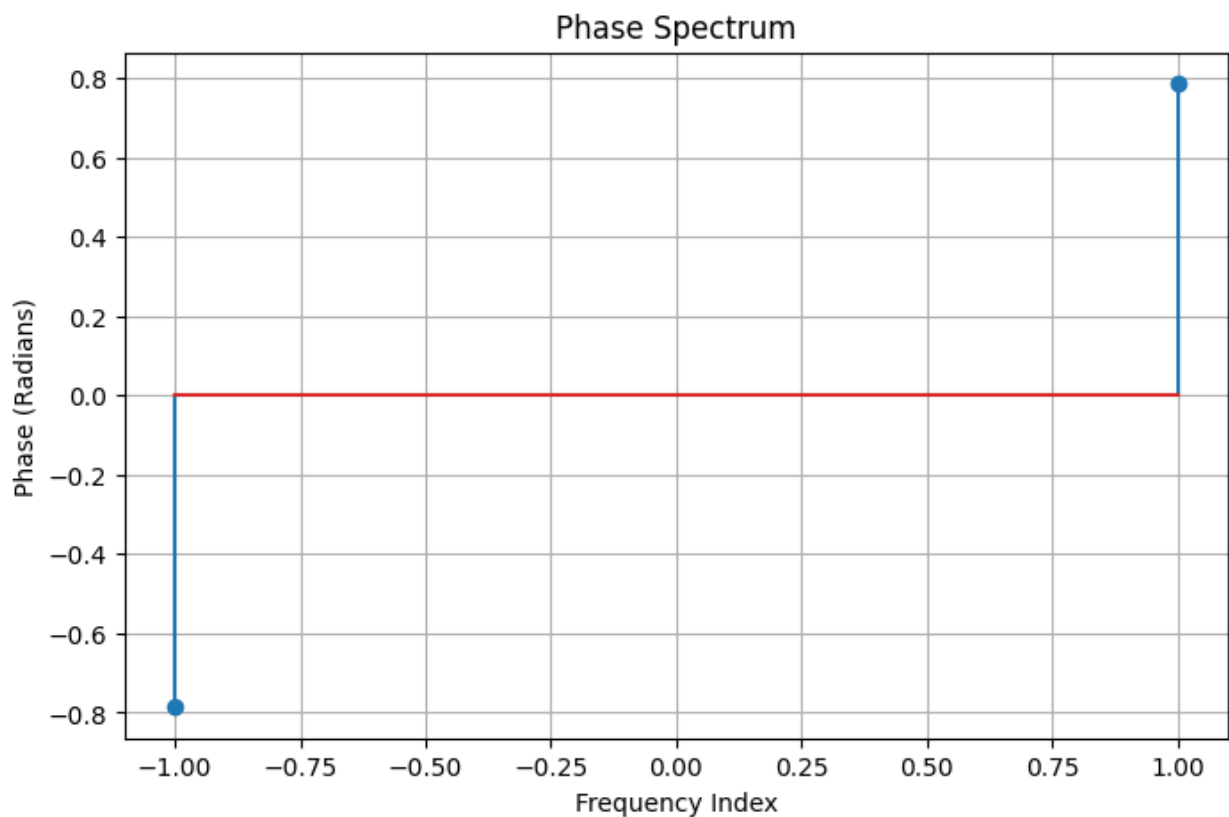
Problem 10

```
frequency = np.array([-1,1])
phase = np.array([-np.pi/4,np.pi/4])
plt.figure(figsize=(8,5))
plt.stem(frequency,phase)
plt.title("Phase Spectrum")
```

```

plt.xlabel("Frequency Index")
plt.ylabel("Phase (Radians)")
plt.grid(True)
plt.show()
print("Positive Frequency Phase :", np.pi/4)
print("Negative Frequency Phase :", -np.pi/4)

```



```

Positive Frequency Phase : 0.7853981633974483
Negative Frequency Phase : -0.7853981633974483

```

Section 3

```

import numpy as np
import matplotlib.pyplot as plt

# Frequency indices
freq1 = np.array([-1,1])
amp1 = np.array([0.5,0.5])

```

```

freq2 = np.array([-3,-1,1,3])
amp2 = np.array([0.25,0.5,0.5,0.25])

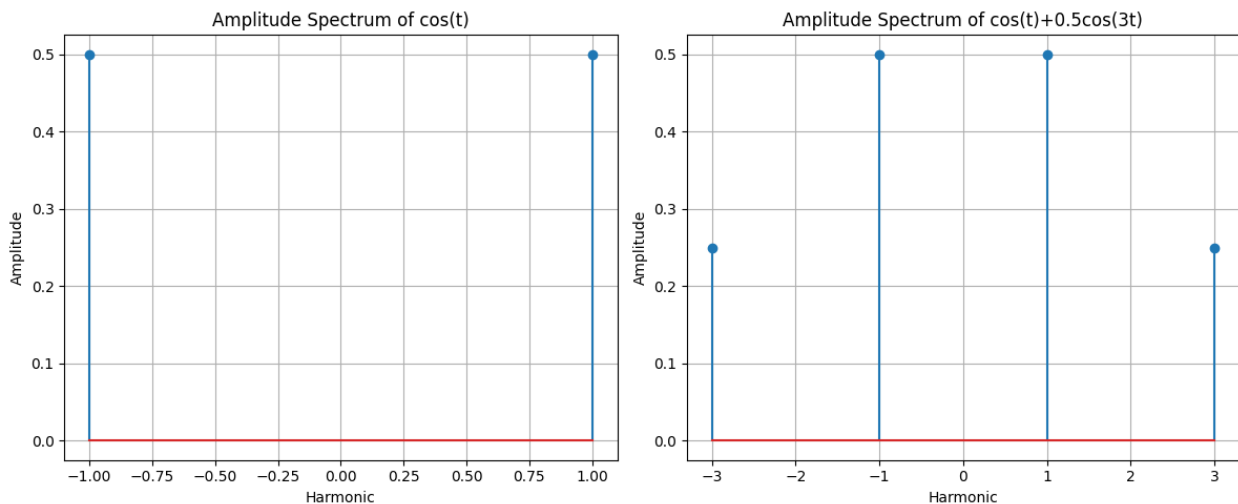
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.stem(freq1,amp1)
plt.title("Amplitude Spectrum of cos(t)")
plt.xlabel("Harmonic")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(1,2,2)
plt.stem(freq2,amp2)
plt.title("Amplitude Spectrum of cos(t)+0.5cos(3t)")
plt.xlabel("Harmonic")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.show()

```



Problem 12

```

t = np.linspace(0,2*np.pi,1000)
x1 = np.cos(t)
x2 = np.sin(2*t)

a = 2
b = 3

```

```

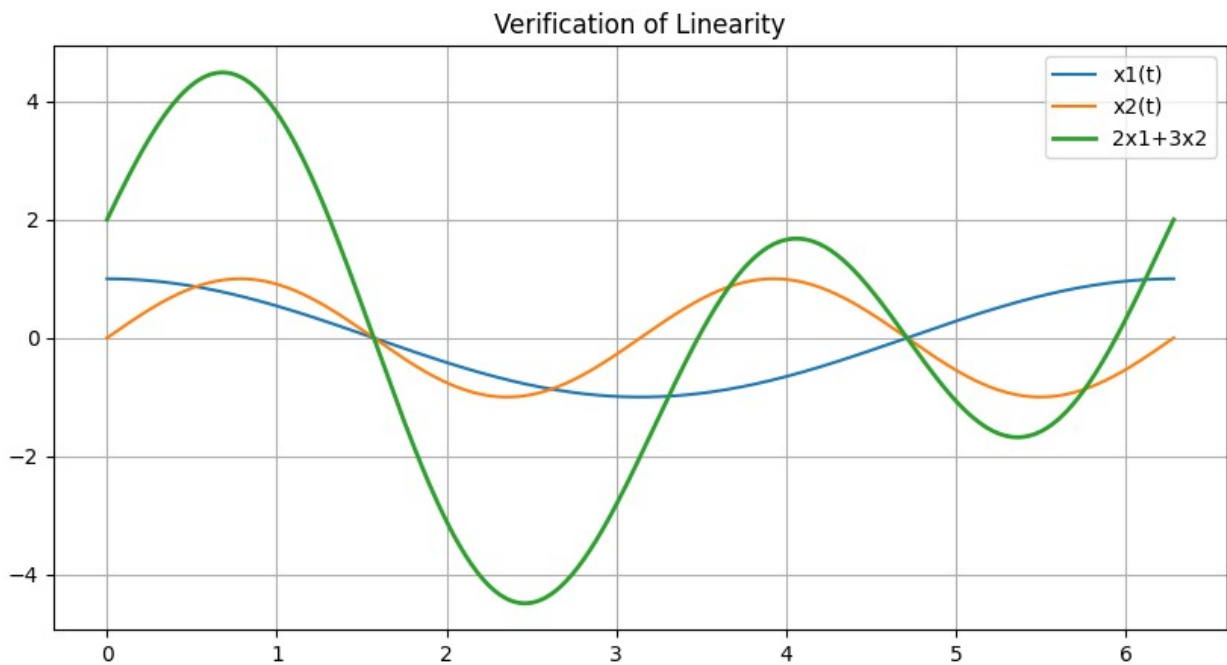
x3 = a*x1 + b*x2

plt.figure(figsize=(10,5))

plt.plot(t,x1,label="x1(t)")
plt.plot(t,x2,label="x2(t)")
plt.plot(t,x3,label="2x1+3x2",linewidth=2)

plt.legend()
plt.grid(True)
plt.title("Verification of Linearity")
plt.show()

```



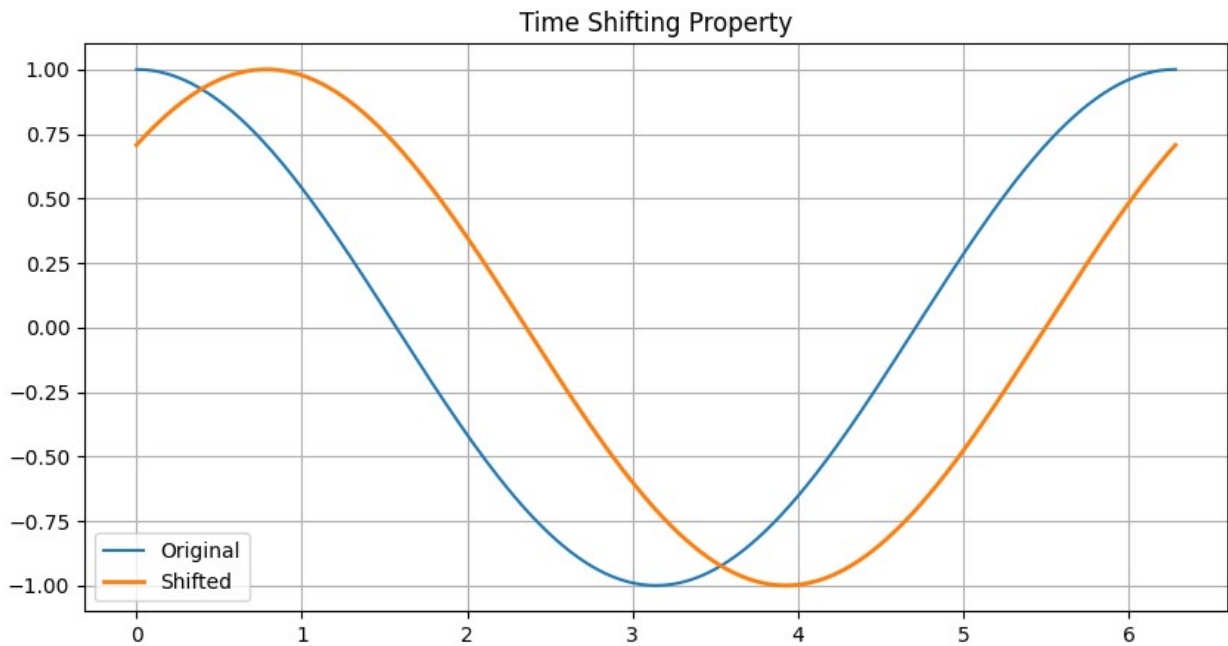
Problem 13

```

t = np.linspace(0,2*np.pi,1000)
x = np.cos(t)
x_shift = np.cos(t-np.pi/4)
plt.figure(figsize=(10,5))
plt.plot(t,x,label="Original")
plt.plot(t,x_shift,label="Shifted",linewidth=2)
plt.legend()

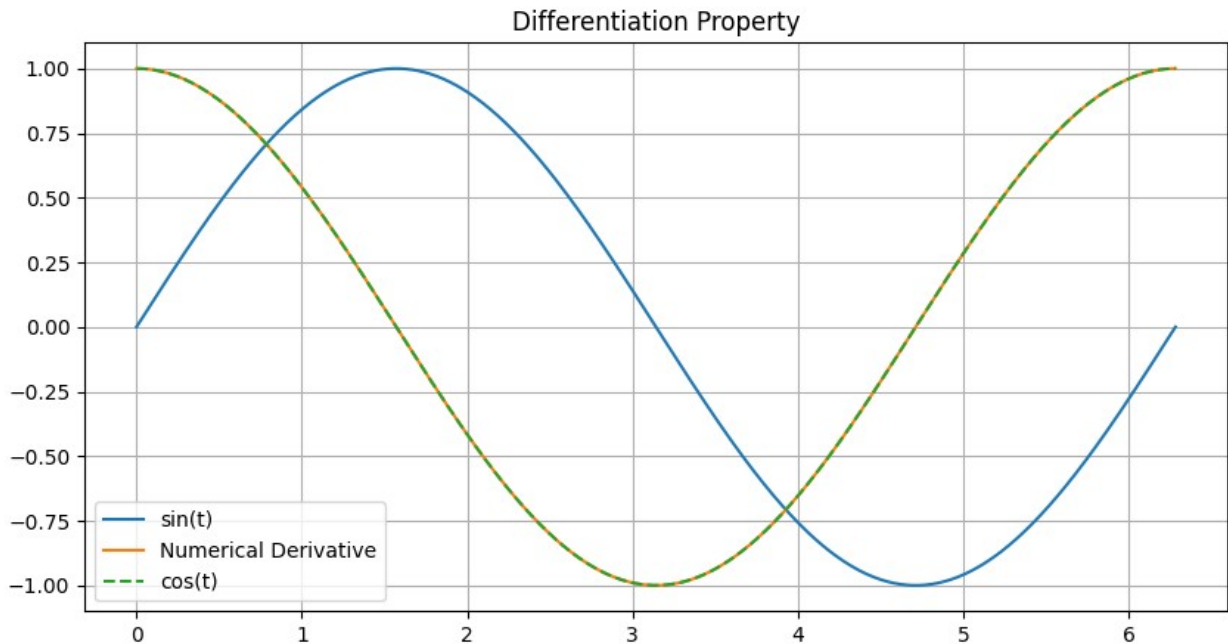
```

```
plt.grid(True)
plt.title("Time Shifting Property")
plt.show()
```



Problem 14

```
t = np.linspace(0, 2*np.pi, 1000)
x = np.sin(t)
dx = np.gradient(x, t)
plt.figure(figsize=(10, 5))
plt.plot(t, x, label="sin(t)")
plt.plot(t, dx, label="Numerical Derivative")
plt.plot(t, np.cos(t), '--', label="cos(t)")
plt.legend()
plt.grid(True)
plt.title("Differentiation Property")
plt.show()
```



Problem 15

Average Energy (Time Domain) = 0.5

Energy from Fourier Coefficients = 0.5

Challenge Problems

Challenge Problem 1

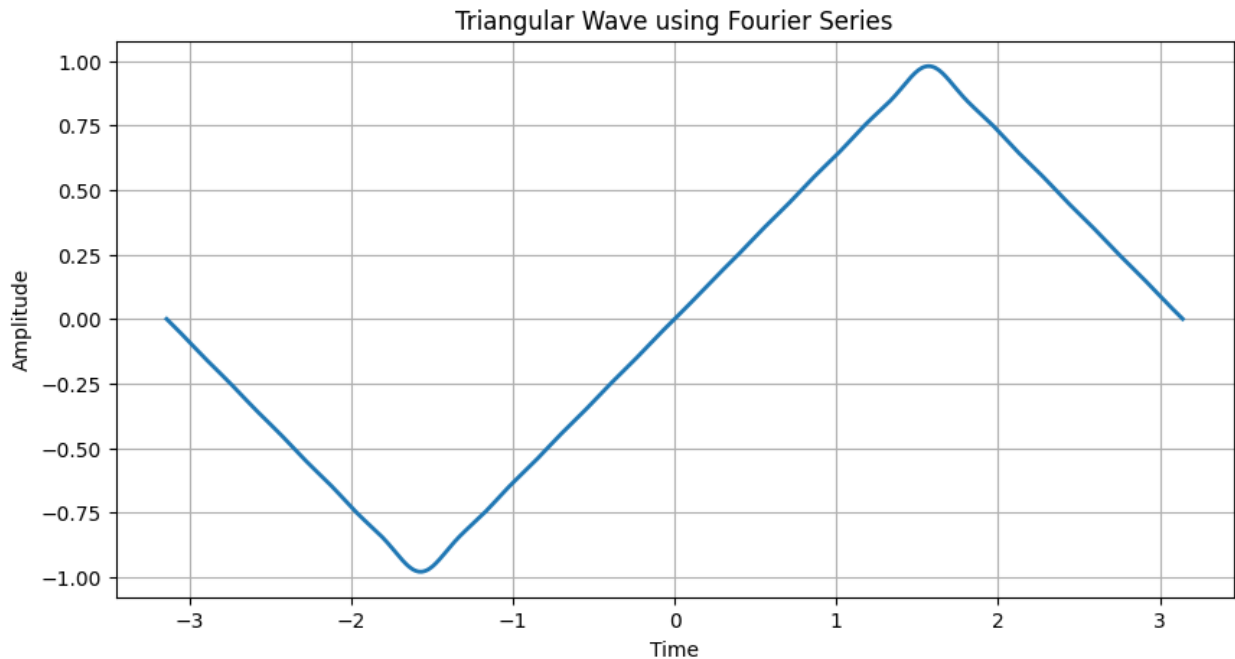
```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-np.pi, np.pi, 1000)

triangle = np.zeros_like(t)

for n in range(1,21,2):
    triangle += ((-1)**((n-1)//2))*(8/(np.pi**2*n**2))*np.sin(n*t)

plt.figure(figsize=(10,5))
plt.plot(t, triangle, linewidth=2)
plt.title("Triangular Wave using Fourier Series")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

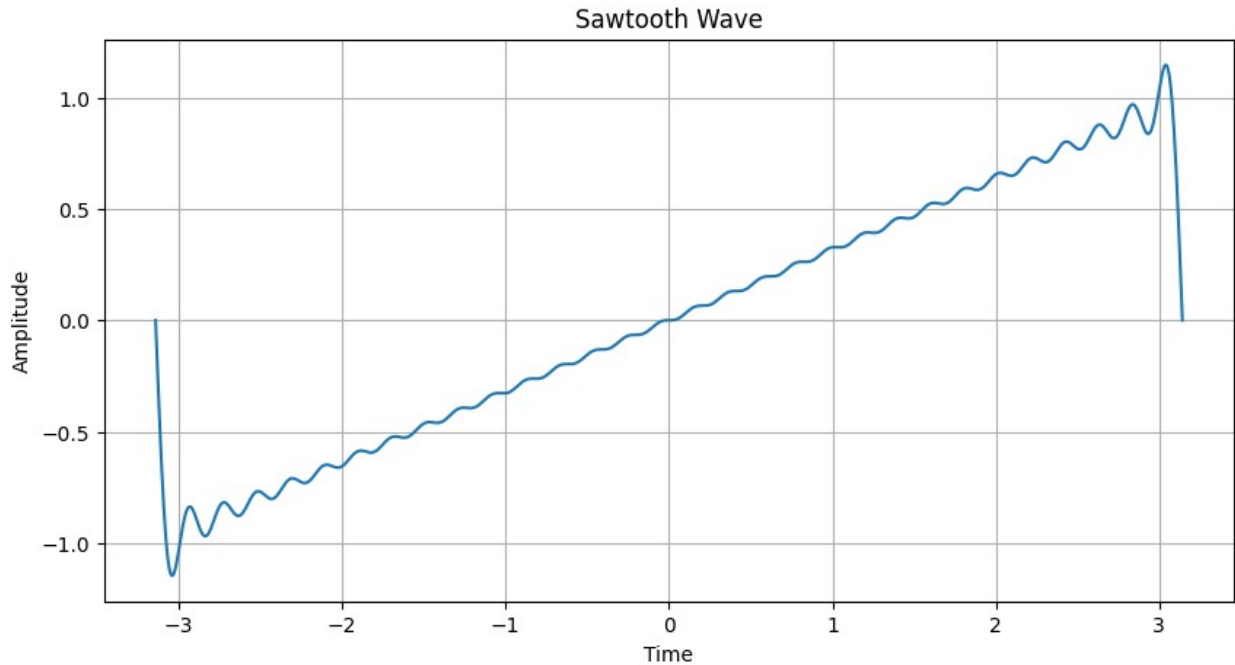


Challenge Problem 2

```
t = np.linspace(-np.pi,np.pi,1000)
saw = np.zeros_like(t)

for n in range(1,31):
    saw += ((-1)**(n+1))*2/(np.pi*n)*np.sin(n*t)

plt.figure(figsize=(10,5))
plt.plot(t,saw)
plt.title("Sawtooth Wave")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```



Challenge Problem 3

```
import numpy as np
import matplotlib.pyplot as plt

def fourier_series(coefficients, t):
    """
    coefficients : dictionary
    key = harmonic number
    value = amplitude
    """

    x = np.zeros_like(t)

    for n, a in coefficients.items():
        x += a*np.cos(n*t)

    return x

t = np.linspace(-np.pi, np.pi, 1000)

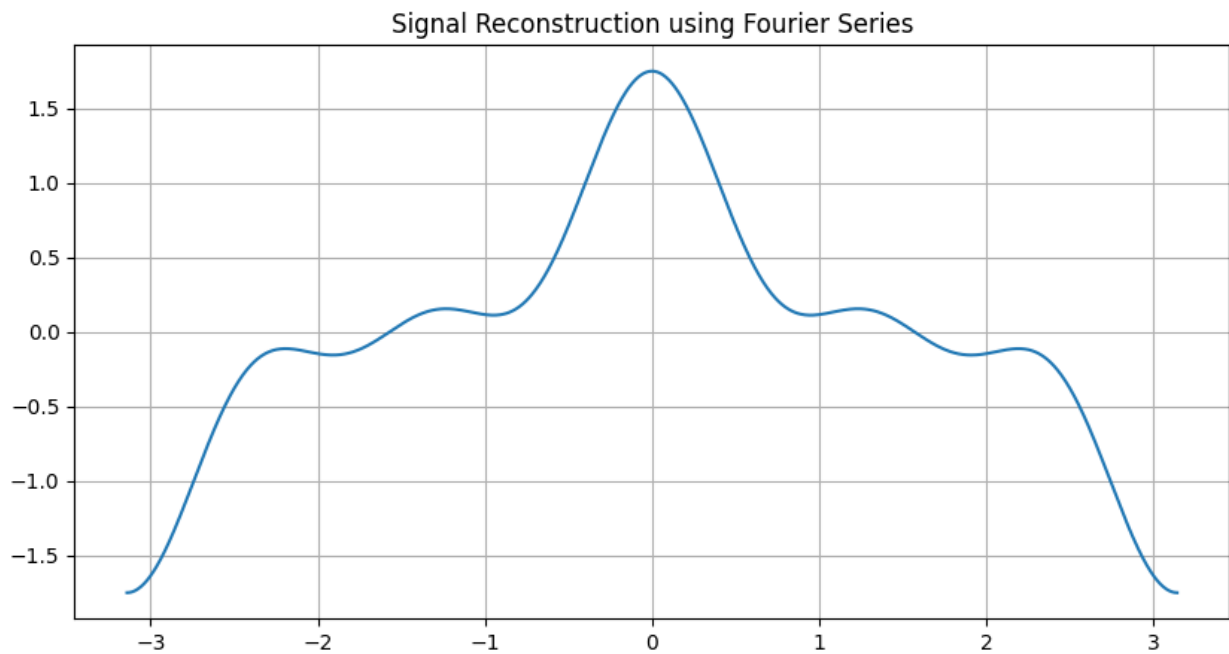
coeff = {
    1:1,
    3:0.5,
    5:0.25
}

signal = fourier_series(coeff,t)

plt.figure(figsize=(10,5))
```

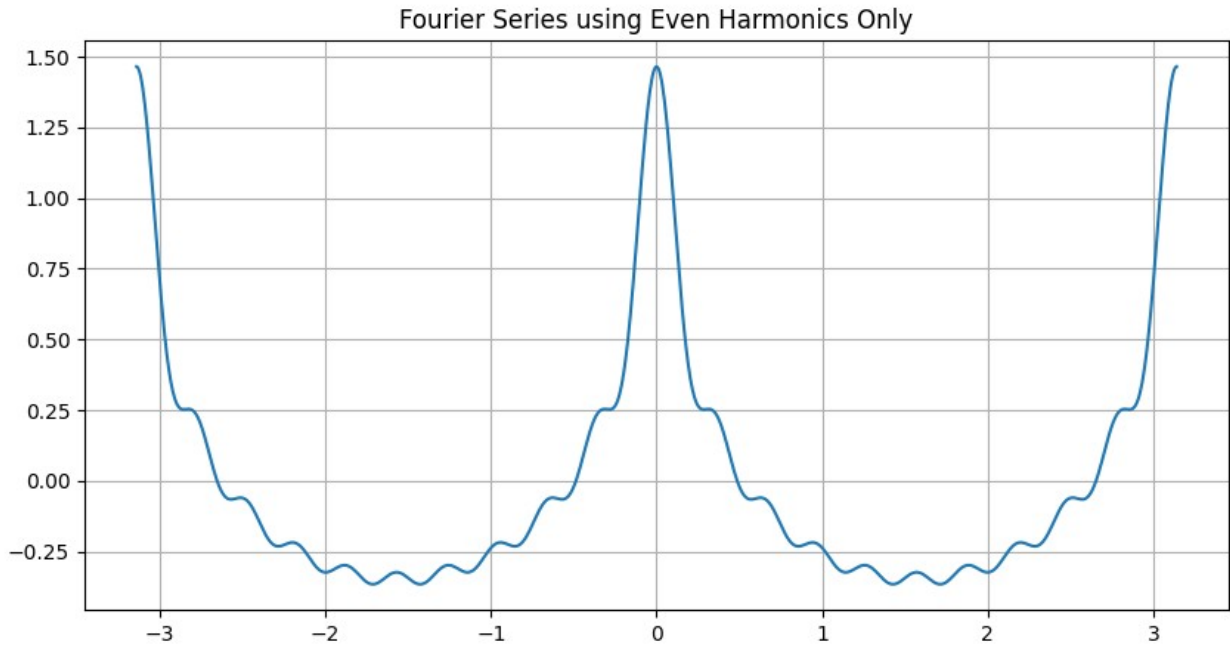


```
plt.plot(t,signal)
plt.title("Signal Reconstruction using Fourier Series")
plt.grid(True)
plt.show()
```



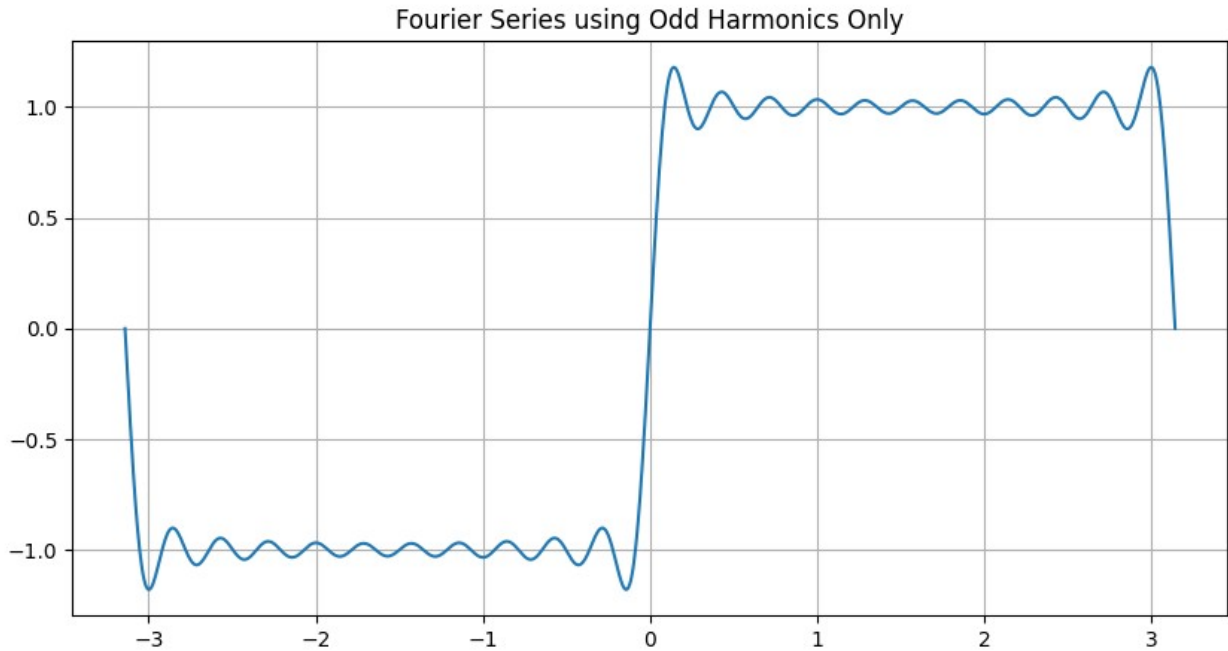
Challenge Problem 4

```
t = np.linspace(-np.pi,np.pi,1000)
x = np.zeros_like(t)
for n in range(2,22,2):
    x += (1/n)*np.cos(n*t)
plt.figure(figsize=(10,5))
plt.plot(t,x)
plt.title("Fourier Series using Even Harmonics Only")
plt.grid(True)
plt.show()
```



Challenge Problem 5

```
t = np.linspace(-np.pi,np.pi,1000)
x = np.zeros_like(t)
for n in range(1,22,2):
    x += (4/(np.pi*n))*np.sin(n*t)
plt.figure(figsize=(10,5))
plt.plot(t,x)
plt.title("Fourier Series using Odd Harmonics Only")
plt.grid(True)
plt.show()
```



Challenge Problem 6

```
t = np.linspace(-np.pi,np.pi,1000)
terms = [5,15,25,75]
plt.figure(figsize=(12,10))
for i,N in enumerate(terms):
    x = np.zeros_like(t)
    for n in range(1,N+1,2):
        x += (4/(np.pi*n))*np.sin(n*t)
    plt.subplot(2,2,i+1)
    plt.plot(t,x)
    plt.title(f"{N} Fourier Terms")
    plt.grid(True)
plt.tight_layout()
plt.show()
```

